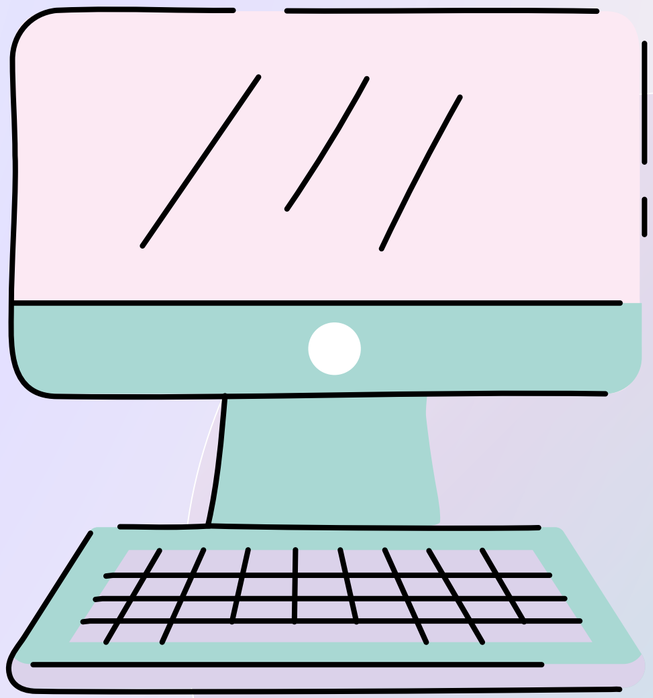
INTELIGENCJA OBLICZENIOWA W ANALIZIE DANYCH CYFROWYCH

Środowisko Gymnasium



Opracowanie:
Julia Kaniuka

UCZENIE ZE WZMOCNIENIEM
(RL - REINFORCEMENT LEARNING)

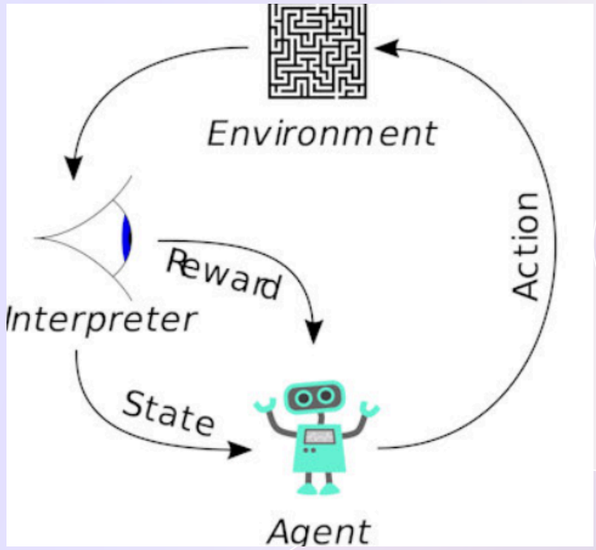
Agent uczy się podejmowania decyzji poprzez interakcję ze Środowiskiem, w którym może bezpiecznie eksperymentować i wyciągać wnioski z konsekwencji swoich działań. Nie dostaje on gotowych odpowiedzi, ale otrzymuje nagrody (rewards) lub kary za podjęte działania.

Kluczowe pojęcia:

- **Agent:** "Mózg" programu, który podejmuje decyzje.
- **Środowisko** (Environment): Świat, w którym żyje agent (np. plansza gry).
- **Stan** (State): Opis aktualnej sytuacji (stanem może być liczba np. index pola w Frozen Lake) lub wektor liczb (np. pozycja i prędkość w Mountain Car).
- **Akcja** (Action): To, co agent może zrobić (akcje mogą być dyskretne (np. 0-lewo, 1-dół) lub ciągłe (np. siła nacisku na pedał gazu).
- **Nagroda** (Reward): Numeryczny sygnał zwrotny (np. +1 za dotarcie do celu, -10 za wpadnięcie do dziury).

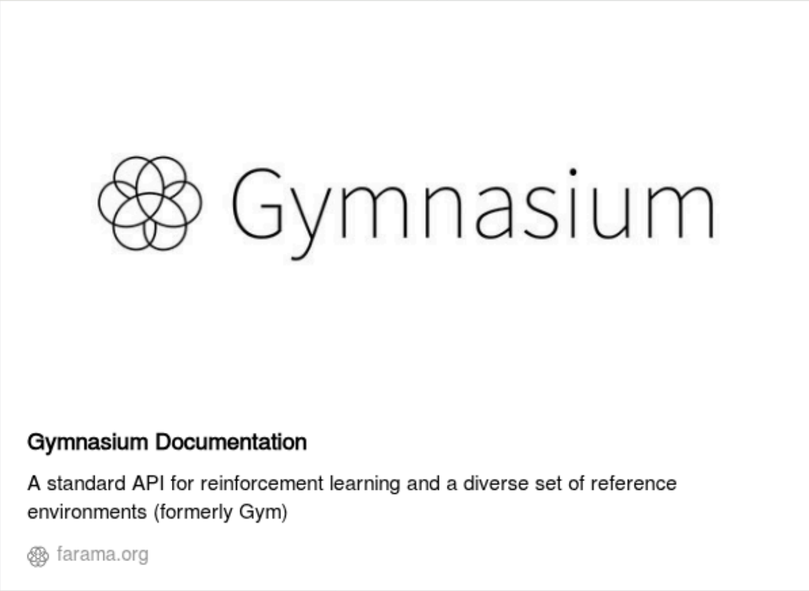
BIBLIOTEKA GYMNASIUM

Gymnasium to biblioteka Python (następca klasycznego Gym), która dostarcza ujednolicony interfejs do testowania algorytmów RL.



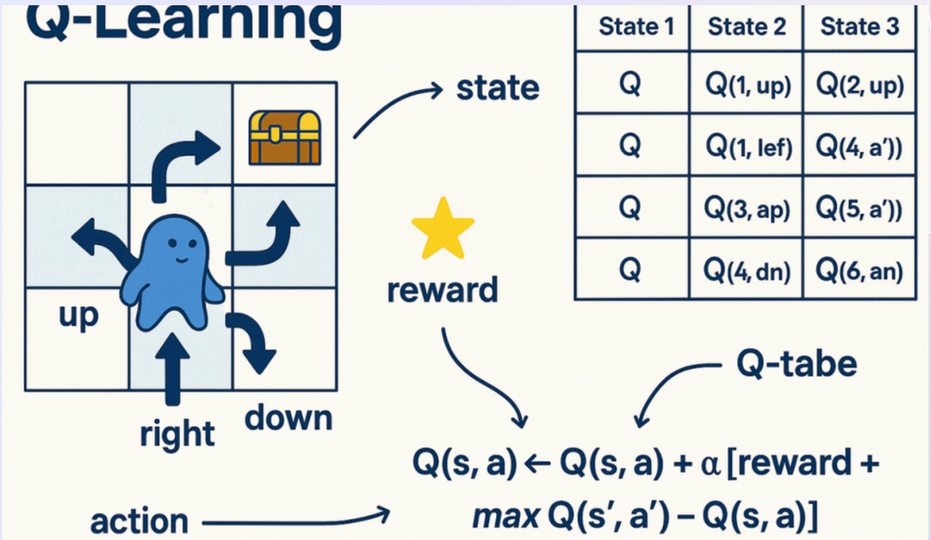
Sercem Gymnasium jest klasa Env.
Cztery kluczowe funkcje tej klasy:

- make() - stworzenie środowiska (wybór gry)
- Env.reset() - resetuje środowisko do stanu początkowego, **wymaganego przed wywołaniem kroku**, zwraca pierwszą obserwację agenta dla epizodu
- Env.step() - aktualizuje środowisko akcjami, zwracając kolejną obserwację agenta, nagrodę za podjęcie tych akcji
- Env.render() - renderuje środowiska, aby pomóc zwizualizować to, co widzi agent.



ALGORYTM Q-LEARNING

Celem algorytmu jest znalezienie polityki optymalnej, czyli zestawu instrukcji: "*jeśli jesteś w sytuacji X, zrób Y, bo to statystycznie przyniesie Ci najwięcej punktów w przyszłości*". Uczy się poprzez próbkowanie doświadczeń i aktualizację swoich wartości Q w oparciu o zaobserwowane nagrody i przejścia.



State 1	State 2	State 3
Q	Q(1, up)	Q(2, up)
Q	Q(1, lef)	Q(4, a'))
Q	Q(3, ap)	Q(5, a'))
Q	Q(4, dn)	Q(6, an)

Q-learning tworzy gigantyczną „ściągę” zwaną macierzą Q, która informuje agenta, jak dobre jest każde działanie w danej sytuacji:

- **Wiersze** = różne sytuacje (stany), z którymi może spotkać się agent
- **Kolumny** = różne działania, które agent może podjąć
- **Wartości** = jak dobre jest dane działanie w danej sytuacji (oczekiwana przyszła nagroda)

HIPERPARAMETRY

Alfa (Learning Rate) - tempo uczenia, decyduje o tym, jak bardzo nowa informacja nadpisuje starą wiedzę.

- **Zbyt wysoka** (np. 0.9): Agent jest "porywczy". Każdy pojedynczy sukces lub porażka drastycznie zmieniają jego zdanie, agent "zapomina" to, co wiedział wcześniej.
- **Zbyt niska** (np. 0.001): Agent jest "uparty". Potrzebuje tysięcy sukcesów, żeby w ogóle zacząć uważać daną drogę za dobrą.
- **Optymalna (0.1):** Stabilny balans między nowym doświadczeniem a zdobytą już wiedzą.

Współczynnik dyskontowy γ (gamma), gdzie γ należy do $[0, 1]$ - określa, jak bardzo agent dba o nagrody, które otrzyma w przyszłości, w stosunku do tych natychmiastowych.

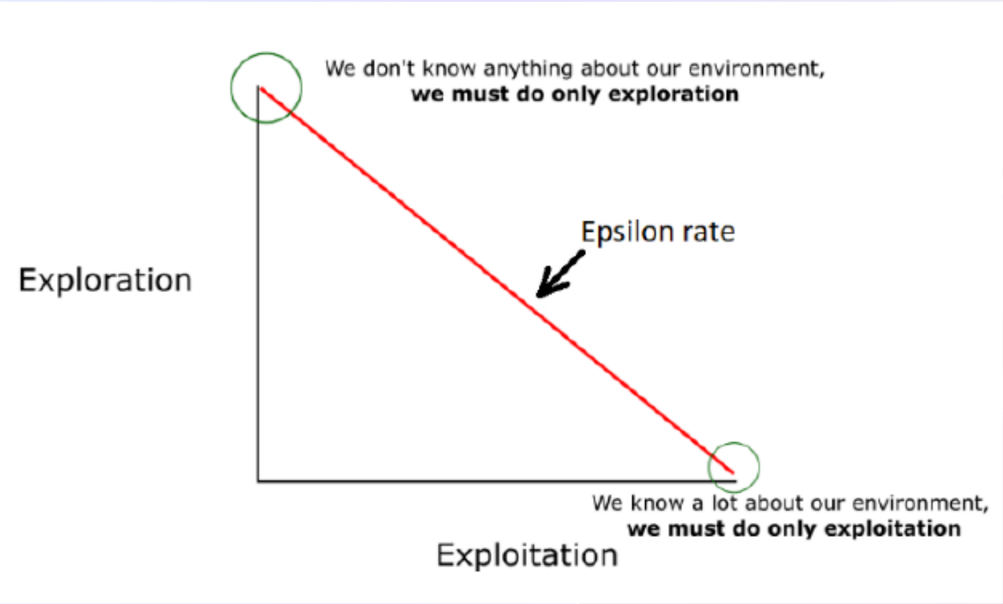
- γ **bliskie 0:** Agent jest "krótkowzroczny". Liczy się tylko to, co dostanie w następnym kroku.
- γ **bliskie 1:** Agent jest "dalekowzroczny". Jest gotowy zrezygnować z małej nagrody teraz, aby zyskać ogromną nagrodę za np. 50 kroków.
- w projekcie γ **= 0.9**, co jest złotym środkiem – agent ceni przyszłość, ale z lekkim dystansem.

HIPERPARAMETRY

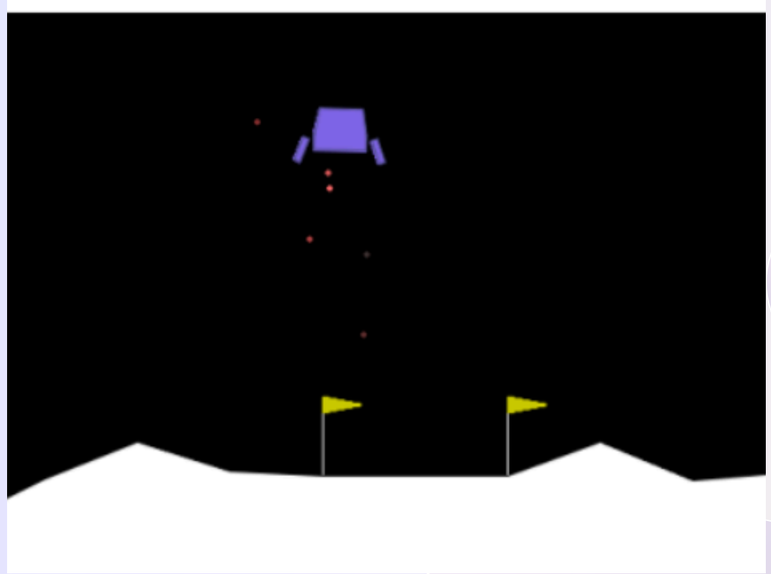
Agent musi balansować między dwoma zachowaniami:

- **Eksploatacja:** Wybieram akcję, która ma najwyższą wartość w tabeli Q (korzystam z wiedzy). Wykorzystaj aktualną wiedzę, aby zdobyć najlepsze nagrody.
- **Eksploracja:** Wybieram losową akcję, aby sprawdzić, czy nie istnieje jeszcze lepsza ścieżka, której jeszcze nie odkryłem. Wypróbuj nowe działania, aby dowiedzieć się więcej o środowisku.

Zazwyczaj używamy **strategii epsilon-greedy (ϵ):**
Z prawdopodobieństwem ϵ: wybierz losową akcję (eksploruj)
Z prawdopodobieństwem 1-ϵ: wybierz najlepszą znaną akcję (eksploatację)



CO JEŚLI PRZESTRZEŃ OBSERWACJI JEST CIĄGŁA?



1. **Dyskretyzacja do tabeli Q***

Polega na nałożeniu "wirtualnej siatki" na ciągłe dane. Każdy z wymiarów dzielimy na określoną liczbę przedziałów (np. prędkość dzielimy na 10 zakresów: "bardzo wolno", "wolno", ..., "bardzo szybko").

2. **Aproksymacja funkcji**

Użycie sieci neuronowej. Sieć nie przechowuje wartości Q w komórkach, ale uczy się wzoru (funkcji), który pozwala jej obliczyć nagrodę dla dowolnych liczb wejściowych.

- **DQN** (Deep Q-Network) - sieć analizuje dane wejściowe i przewiduje, która akcja przyniesie najlepszą nagrodę w przyszłości.
- **PPO** (Proximal Policy Optimization) - działa w cyklach: najpierw agent zbiera trochę doświadczenia, a potem na tym zbiorze danych aktualizuje swoją wiedzę.
- **DDPG** (Deep Deterministic Policy Gradient) - używa dwóch sieci neuronowych, które ze sobą współpracują (architektura Aktor-Krytyk).

*Dyskretyzacja wiąże się z utrata dokładności. Na przykład, jeśli podzielimy przestrzeń na zbyt małą liczbę przedziałów, agent może nie rozpoznać ważnych subtelności środowiska. Z kolei, jeśli podzielimy ją na zbyt dużą liczbę przedziałów, algorytm może stać się zbyt kosztowny obliczeniowo (wymaga więcej pamięci i czasu).